# THESIS PROGRESS REPORT

## Zero-Shot Nutrient and Quantity Association using OCR-Guided Semantic Graph Matching

Moustafa Hamada · M.Sc. AI & Data Science
Deggendorf Institute of Technology · University of South Bohemia

Dataset · Pipeline · Experiment 1 Results · Failure Analysis · Planned Experiments

# 1. Thesis Objective

This thesis builds a modular zero-shot information extraction pipeline that converts supplement label images into structured nutritional tuples. The term zero-shot means no labelled training examples are used at any stage — all rules and logic are derived from domain knowledge about German/ EU supplement label structure.

**Target output schema:**

```
<nutrient, quantity, unit, context>
Example: Magnesium, 400, mg, per_100g
```

# 2. Dataset Collection and Annotation

## 2.1 Data Collection

The dataset was collected by manually photographing nutritional supplement labels from supermarkets and health stores beside the provided dataset From the Profesor. Labels were selected to cover a diverse range of product types, languages, and layout formats representative of the German supplement market:

| Property | Details |
|---|---|
| Total images | 43 JPEG supplement label photographs |
| Label languages | German (primary), English, multilingual (DE/EN/FR/NL/IT/ES/PL) |
| Product types | Tablets/capsules, powder sachets, electrolyte powders, multivitamins, liquid supplements |
| Layout formats | Single-context, dual-column (per 100g +/ per serving), multilingual blocks, vitamin tables |
| Collection method | Provided From the Professor + Direct photography of physical product labels in retail environments |

## 2.2 Annotation Process

Annotation was performed in two iterations to ensure accuracy and consistency.

### Iteration 1 — AI-Assisted Annotation (Gemini 3 Pro)

All 43 images were annotated using Gemini 3 Pro via a structured prompt that instructs the model to extract nutritional tuples directly from the label image. The prompt enforces strict extraction rules: values must be explicitly visible, units must not be merged with quantities, nutrient names must be preserved in their original language, and context must be resolved from the nearest applicable header.

The annotation prompt used:

```
SYSTEM / ROLE INSTRUCTION
You are an expert data annotator for a research project on nutritional supplement
label understanding.
Your task is to extract structured nutritional tuples from a supplement label image.
You must strictly follow the annotation rules and output format.
Do NOT invent values.
```

```
Do NOT normalize units unless specified.
Only extract information that is explicitly visible in the image.
◇ TASK DESCRIPTION
Given a supplement label image, extract all structured entries that contain:

Nutrient name (text entity)
Quantity (numeric value only)
Unit (e.g., mg, µg, g)
Context (e.g., per_daily_dose, per_serving, per_100g)
NRV Percentage (numeric value only, if present)
Serving Size (text or numeric with unit, if explicitly stated for the given context,
e.g., "2 Kapseln", "70g")
Extract entries from:

Nutrient tables
Structured ingredient sections that contain numeric values
Ignore:

Marketing claims
Descriptive text paragraphs
Instructions
Non-quantified ingredients
Percentages unless they are paired with a quantity or explicitly belong to the %NRV
column.
◇ CONTEXT RESOLUTION RULES
If the label contains:

"Je Tagesdosis" → context = per_daily_dose
"Pro Portion" → context = per_serving
"Per serving" → context = per_serving
"Pro 100 g" → context = per_100g
If multiple contexts exist, assign each nutrient to the closest applicable header.
◇ EXTRACTION RULES

Extract entries exactly as written.
Preserve nutrient names in the original language.
Quantity must contain only the numeric value.
Unit must be separate from quantity.
Do not merge rows.
If breakdown entries exist (e.g., Magnesiumoxid, Magnesiumcitrat), extract them as
separate tuples.
If "(gesamt)" appears, keep it in the nutrient name.
Extract the %NRV value if present in the row. Strip the '%' sign and output only the
number.
Extract the serving size associated with the context if visible on the label (e.g.,
4g, 70g, 2 Kapseln).
If no NRV or serving size is present/applicable for a specific row (e.g., per_100g
usually has no specific serving size), leave the respective field empty/blank.
Do not compute derived values.
◇ OUTPUT FORMAT (STRICT)
Return ONLY CSV format.
Header must be
exactly:image_id,nutrient,quantity,unit,context,nrv_percent,serving_size
Each row must
follow:<image_id>,<nutrient>,<quantity>,<unit>,<context>,<nrv_percent>,<serving_size>
I am giving image by image.
```

Each image was submitted individually. Gemini returned one CSV row per extracted nutritional entry.

### Iteration 2 — Manual Review and Correction

Every annotation produced in Iteration 1 was manually reviewed entry by entry. The review process:

- Verified that all visible nutrient rows were captured — missing rows were added manually
- Corrected misassigned context values
- Fixed nutrient name errors — OCR-like confusions in Gemini output were corrected to match the label exactly
- Validated numeric values against the physical label — quantities incorrectly merged with units were separated
- Removed hallucinated entries — any row not explicitly visible on the label was deleted.

The final annotation schema for Experiment 1 uses a 4-field subset of the full schema (nutrient, quantity, unit, context), with NRV percent and serving size retained in the master file but excluded from pipeline evaluation targets.

| Schema version | Fields | Used for |
|---|---|---|
| Full (master) | image_id, nutrient, quantity, unit, context, nrv_percent, serving_size | Storage and reference |
| 4-field (active) | image_id, nutrient, quantity, unit, context | Pipeline evaluation — Experiments 1-4 |

**Note** *The 4-field schema contains 595 ground truth tuples across 43 images.*

# 3. Pipeline Architecture

The pipeline has 6 sequential stages. Each stage is a standalone Python module. No stage depends on the internal implementation of another — this modularity enables swapping individual components for controlled experiments.

| Stage | File | What It Does |
|---|---|---|
| 1 | src/ocr/ocr_runner.py | Runs EasyOCR on the label image. Returns a list of text tokens, each with bounding box coordinates and confidence score. The only stage that touches pixel data. |
| 2 | src/utils/ocr_corrector.py | Cleans OCR output before classification. Fixes systematic character confusions (O->0, l->1, rn->m, Ug->µg), strips table-border artefacts from token edges, and applies fuzzy matching to snap corrupted nutrient names to the correct lexicon entry. |

| Stage | File | What It Does |
|---|---|---|
| 3 | src/classification/experiment_01_final_semantic_classifier.py | Assigns one of five labels to each token: NUTRIENT, QUANTITY, UNIT, CONTEXT, NOISE. Uses a priority rule chain — NOISE first to prevent false positives, then UNIT, CONTEXT, QUANTITY, NUTRIENT. Handles fused tokens (400mg -> QUANTITY:400 + UNIT:mg), slash-variant multilingual names (Fett/fat -> NUTRIENT), and vitamin shorthands (B12 -> NUTRIENT). Context tokens are immediately normalised to canonical form (per_100g, per_serving, per_daily_dose). |
| 4 | src/graph/graph_constructor.py | Builds a typed semantic graph over the labelled tokens. Four edge types: SAME_ROW (tokens on the same visual line), SAME_COL (tokens in the same vertical column), ADJACENT (tokens physically next to each other), CONTEXT_SCOPE (context header governs nutrients below it). The typed graph encodes semantic layout relationships — this is the core scientific contribution. |
| 5 | src/matching/experiment_01_final_association.py | Traverses the graph to extract tuples. For each NUTRIENT node: finds quantities on the same row (sorted left-to-right for multi-column disambiguation), falls back to column search, resolves context per-quantity to correctly assign per_100g vs per_serving in multi-column tables. Includes block inheritance for labels where multiple nutrient rows share one quantity and unit. |

| Stage | File | What It Does |
|---|---|---|
| 6 | src/evaluation/evaluator.py | Compares predictions against ground truth. Computes Nutrient F1, Quantity/Unit/Context match accuracy, and Full Tuple F1 (all four fields correct). Saves per-image breakdown, error analysis CSV, and pipeline diagnostics CSV (token and edge counts per stage per image). |
| Orch. | run_experiment.py | Orchestrates the full experiment: discovers images, runs all pipeline stages, collects per-stage diagnostics, saves tuples CSV, calls the evaluator, and prints a comparison delta against a previous experiment. |

# 4. Experiment 1 — Current Results

Experiment 1 establishes the baseline using EasyOCR and the rule-based semantic graph pipeline. Evaluated on 43 images against 595 ground truth tuples (4-field schema).

## 4.1 Metrics — Official Baseline

| Metric | Value | Notes |
|---|---|---|
| Nutrient Recall | 34.5% | Of all GT nutrients, fraction the pipeline found |
| Nutrient F1 | 0.465 | Harmonic mean |
| Quantity Match Acc | 26.3% | Among matched nutrient pairs |
| Unit Match Acc | 62.9% | Among matched nutrient pairs |
| Context Match Acc | 29.3% | Among matched nutrient pairs |
| Full Tuple Precision | 5.2% | Of all predicted tuples, fraction with all 4 fields correct |
| Full Tuple Recall | 2.5% | Of all 595 GT tuples, fraction fully correctly extracted |
| ★ Full Tuple F1 | 3.4% | Primary thesis metric — end-to-end extraction quality |

**Note** *Full Tuple F1 is the correct primary metric — a tuple is correct only when all four fields (nutrient, quantity, unit, context) match ground truth simultaneously.*

## 4.2 Fix Progression Applied During Experiment 1

| Fix | What Changed | Metric Impact |
|---|---|---|
| Fix 1 | norm_context() translation map — DE/EN context variants mapped to canonical underscore form | Context Acc: 0% -> 1.9% |
| Fix 2 | CONTEXT_SCOPE edge construction — removed incorrect horizontal overlap check that prevented all context edges from being built | Context Acc: 1.9% -> 15.0% |
| Fix 3 | Quantity x-sort (left->right) + per-quantity context lookup — fixes multi-column per_100g/per_serving assignment | Qty Acc: 18% -> 27.2% |
| Fix 4 | Expanded CONTEXT_MAP with discovered OCR variants (PRO PORTION, Je 1009, etc.) | Context Acc: 15% -> 27.7% |
| Fix 5 | CONTEXT_MAP moved from evaluator into classifier — context normalised at classification time. Also Removed a duplicated image entry from the GT csv file | Context Acc: 27.7% -> 29.3% |
| Fix 6 | NUTRIENT lexicon expansion — English fields, slash-variants, shorthands, German/Dutch additions | Nutrient F1: 0.465 -> 0.486, +16 matched pairs |

# 5. Failure Analysis — Where Tuples Are Lost

Pipeline diagnostics across 43 images reveal five root causes for the 97.5% of GT tuples not yet correctly extracted:

## 5.1 Loss Funnel

| Stage | Failure | GT Tuples Lost | Fixable in Exp 1? |
|---|---|---|---|
| OCR | 5 images return zero tokens (4, 12, 16, 26, 41) | 48 tuples | No — requires better OCR |
| OCR | 36.2% of tokens below confidence threshold — many images near-100% NOISE | ~100 tuples | Partially — Experiment 2 |
| Classifier | English-only nutrient names not in German-first lexicon (DIETARY FIBRE, SODIUM, etc.) | ~80 tuples | Yes — Fix 6 applied |
| Classifier | Multilingual slash-variant names too long for slash-split (12, 201) | ~30 tuples | Partially — token serialiser needed |

| Stage | Failure | GT Tuples Lost | Fixable in Exp 1? |
|---|---|---|---|
| Classifier | Context header not recognised — clf_context=0 for 7 images | ~120 tuples | Partially — Fix B1 applied |
| Graph | Context tokens exist but CONTEXT_SCOPE edges point to wrong column | ~40 tuples | Experiment 4 — TCA |
| Association | Block inheritance missing — rows sharing one qty+unit produce qty=None | ~30 tuples | Experiment 4 — block inheritance |
| Association | Trailing digit OCR artefact: 0.8 -> 0.89, 86.1 -> 86.19 | ~40 tuples | OCR corrector fix pending |

## 5.2 Key Insight — Block Inheritance Pattern

A recurring label structure groups multiple nutrient rows into a measurement block where only the first row carries the quantity and unit:

| Pattern | Example on label | Current pipeline output |
|---|---|---|
| Same nutrient, multiple languages | Magnesium 400mg \| Magnésium \| Magnesio \| Magnesij | Row 1 correct, rows 2-4: qty=None, unit=None |
| Chemical / common name | Vitamin B12 2.5µg \| Cobalamin | Row 1 correct, row 2: qty=None, unit=None |
| Sub-nutrient grouping | Magnesium (gesamt) 400mg \| Magnesiumcitrat \| Magnesiumoxid | Row 1 correct, rows 2-3: qty=None, unit=None |

Fix (Experiment 4): when a nutrient has no findable quantity after all search levels, search upward for the nearest nutrient row above that does have a quantity. Inherit its quantity and unit within a maximum vertical distance (80px ~ 3-4 label rows) to prevent cross-block contamination. Context is always re-resolved from the current row, not inherited.

# 6. Planned Experiments

The thesis follows a controlled ablation design. Each experiment changes one variable while keeping all others fixed — enabling clean decomposition of metric improvements into their sources.

## 6.1 Controlled Variable Design

| Experiment | Variable Changed | Everything Else | Scientific Question |
|---|---|---|---|
| Experiment 1 | Baseline — EasyOCR + rule-based graph | — | What is achievable zero-shot? |
| Experiment 2v1 | OCR engine: EasyOCR -> PaddleOCR | Graph pipeline unchanged | How much failure is OCR-dependent? |
| Experiment 2v2 | OCR engine: EasyOCR -> GLM-OCR (VLM) | Graph pipeline unchanged | Does vision-language OCR help? |
| Experiment 3 | Classifier: rule-based -> GLiNER span extractor | Best OCR from Exp 2 | What does better classification add? |
| Experiment 4 | Graph: TCA + soft edges + block inheritance | Best OCR + classifier | What does better graph reasoning add? |

## 6.2 Experiment 2 — OCR Engine Swap

Two OCR engines will be evaluated as direct replacements for EasyOCR. The semantic graph pipeline is unchanged — only the OCR runner module is swapped.

### Version 1 — PaddleOCR

PaddleOCR 3.0 is not a traditional OCR engine comparable to EasyOCR. PP-OCRv5, its core recognition pipeline, achieves a 1-EditDist score of 0.804 — ranking first across all evaluated systems including Qwen2.5-VL-72B (0.781), Gemini 2.5 Pro (0.768), InternVL3-78B (0.744), and GPT-4o (0.692) — while using only 0.07B parameters, approximately 1/1000th the size of Qwen2.5-VL-72B.

Architecture distinguishing it from EasyOCR: PP-OCRv5 uses PP-HGNetV2 as backbone with knowledge distillation from the GOT-OCR2.0 visual encoder (a large model acting as teacher). Its dual-branch recogniser (GTC-NRTR attention branch + SVTR-HGNet CTC branch) is trained with guidance from a 424B VLM (ERNIE-4.5-VL) for data annotation and filtering. A single model supports Simplified Chinese, Traditional Chinese, Chinese Pinyin, English, and Japanese. Additionally, PP-StructureV3 provides end-to-end document parsing including layout detection, table recognition, and formula recognition.

Expected improvements for supplement labels: significantly better handling of small-font dense text (vitamin tablets 107, 108), robust multilingual recognition on slash-variant labels, elimination of the trailing-digit artefacts (101, 102, 109), and recovery of near-100% NOISE images (8, 15, 30).

### Version 2 — GLM-OCR / GLM-4V Vision-Language Model

#### Motivation

Traditional OCR engines (EasyOCR, PaddleOCR) are text detector + sequence recogniser architectures. They process the image as a sequence of independent text regions without understanding document structure holistically. Vision-language models approach the problem

differently — they encode the full image and generate text conditioned on global visual context, enabling contextual disambiguation that character-level recognisers cannot perform.

### Architecture

GLM-OCR is a vision-language transformer that can be prompted to extract structured text from images. Unlike traditional OCR pipelines, it can be instructed to extract all nutrition label text with bounding box coordinates. The model understands label structure holistically and can resolve ambiguities that confuse character-level recognisers — for example, reading a multilingual slash-variant name as one coherent entity rather than independent characters.

### Implementation Challenge — Bounding Box Availability

The critical challenge is that VLMs may not return per-token bounding boxes. The semantic graph pipeline requires spatial coordinates for every token. Two mitigation approaches:

- Approach A — Direct coordinate extraction: if GLM-OCR can be prompted to return bounding box coordinates alongside text (some model configurations support this), use them directly. Output schema identical to EasyOCR/PaddleOCR.
- Approach B — Hybrid alignment: use GLM-4V for text quality and EasyOCR/PaddleOCR for spatial coordinates. For each GLM-4V token, find the nearest EasyOCR token by character similarity (fuzzy string matching) and assign its bounding box. This decouples OCR recognition quality from spatial positioning.

### OCR Engine Comparison

| Property | EasyOCR | PaddleOCR 3.0 / PP-OCRv5 | GLM-OCR |
|---|---|---|---|
| Architecture | CRAFT detector + CRNN recogniser | PP-HGNetV2 + dual-branch GTC-NRTR/SVTR-HGNet with VLM knowledge distillation | Vision-language transformer (full image encoding) |
| Contextual understanding | None — character-level | None — character-level (distilled from large VLM) | Yes — document-level |
| Bounding boxes | Per-token | Per-token | Not guaranteed — requires alignment step |
| Inference speed | ~0.5s / image | ~0.3s / image | 5–20s / image |
| Pipeline compatibility | Direct | Direct | Requires hybrid alignment |

## 6.3 Experiment 3 — GLiNER Semantic Classifier

GLiNER is a zero-shot span-level named entity recognition model. Instead of classifying individual tokens, it extracts multi-token spans and assigns entity types specified as natural language strings at inference time — no fine-tuning required.

Why it addresses Experiment 1 failures:

- 'davon gesättigte Fettsäuren' — three tokens, classified independently as UNKNOWN+UNKNOWN+NUTRIENT in Exp 1. GLiNER extracts the full span as one NUTRIENT entity.
- Novel compound names not in the lexicon — GLiNER uses contextual embeddings, not lexicon lookup.
- Uppercase/lowercase variants (ENERGIE vs Energie) — bidirectional encoder handles case variation naturally.

The pretrained model 'urchade/gliner_multi-v2.1' supports multilingual documents and can identify supplement label entities without domain-specific training. The context normalisation (CONTEXT_MAP) and token splitter are preserved from Experiment 1.

## 6.4 Experiment 4 — Graph Architecture Improvements (SPADE + PICK inspired)

Two papers from the document information extraction literature directly inform the Experiment 4 improvements:

- SPADEs (Hwang et al., 2021; doi: 10.48550/arXiv.2106.14624) — Spatial Dependency Parsing for semi-structured document understanding. Proposes typed directed edges between OCR tokens, Tail Collision Avoidance (TCA) to resolve competing node assignments, and token serialisation via rel-s edges for compound span reconstruction.
- PICK (Yu et al., 2020, doi: https://doi.org/10.48550/arXiv.2004.07464) — Processing Key Information Extraction from Documents using improved Graph Learning-Convolutional Networks. PICK is a KIE framework that takes OCR output (text + bounding boxes + image segments) and learns a soft adjacency matrix A between graph nodes — replacing manually predefined edge rules with continuous pairwise relationship weights trained end-to-end on labelled documents. It achieves +14.7% F1 over BiLSTM+CRF on medical invoice KIE. PICK requires labelled training data and cannot be applied zero-shot to the supplement label domain. The thesis borrows specifically the soft adjacency concept: the hard 25px SAME_ROW threshold is replaced by a continuous spatial distance weight — a zero-shot approximation of PICK's learned A matrix that requires no training data.

| Improvement | Problem It Solves | Source |
|---|---|---|
| Tail Collision Avoidance (TCA) | Two nutrients on adjacent rows both claim the same quantity — TCA removes the weaker edge so only the correct nutrient keeps it | SPADEs |
| Soft SAME_ROW weighting | Hard 25px row threshold is brittle — replace with continuous distance weight so nearer tokens are preferred. Inspired by PICK's soft adjacency matrix concept: instead of binary edge existence, assign continuous weights reflecting spatial | SPADEs (Hwang et al., 2021) + PICK (Yu et al., 2020) |

| Improvement | Problem It Solves | Source |
|---|---|---|
| | distance. Zero-shot adaptation: no training required. | |
| Token serialisation | Compound nutrient names split across tokens by OCR — merge horizontally adjacent tokens before classification | SPADEs rel-s edges |
| Block inheritance | Multiple nutrient rows share one quantity+unit — inherit qty+unit from nearest block header above within 80px | Novel pattern, Exp 1 analysis |
| NUTRIENT lexicon expansion | English-only and Dutch labels produce zero NUTRIENT tokens — lexicon extended with EU label field names | Exp 1 error analysis |

# 7. Current Status and Next Steps

## 7.1 Completed

- Full pipeline implemented and tested across 43 images (Stages 1–6)
- Experiment 1 baseline established: Full Tuple F1 = 3.4%, Nutrient F1 = 0.465
- Six iterative fixes applied during Experiment 1 development
- v4 classifier refactor: CONTEXT_MAP moved from evaluator into classifier
- v3 evaluator refactor: full tuple definition corrected (all 4 fields required), precision/recall/F1 added
- Pipeline diagnostics system and Deep error analysis: five failure categories documented with root causes and fix strategies
- NUTRIENT lexicon expansion: English fields, slash-variants, shorthand vitamins, German/Dutch additions

## 7.2 Immediate Next Steps for Experiment 2

- Set up PaddleOCR environment and run Experiment 2v1
- Compare Experiment 2v1 vs Experiment 1 baseline — quantify OCR contribution
- Evaluate GLM-OCR feasibility for Experiment 2v2

# 8. References

## 8.1 OCR and Document Parsing

Cui, C., Sun, T., Lin, M., Gao, T., Zhang, Y., Liu, J., Wang, X., Zhang, Z., Zhou, C., Liu, H., Zhang, Y., Lv, W., Huang, K., Zhang, Y., Zhang, J., Zhang, J., Liu, Y., Yu, D., and Ma, Y. (2025). *PaddleOCR 3.0 Technical Report.* doi:10.48550/arXiv.2507.05595

GLM Team (2025). *GLM-4V: Visual Language Model for Document Understanding*. doi: https://doi.org/10.48550/arXiv.2603.10910

## 8.2 Document Information Extraction and Graph Reasoning

Yu, W., Lu, N., Qi, X., Gong, P., and Xiao, R. (2020). *PICK: Processing Key Information Extraction from Documents using Improved Graph Learning-Convolutional Networks.* arXiv:2004.07464. doi: https://doi.org/10.48550/arXiv.2004.07464

Hwang, W., Kim, J., Yim, M., Seo, I., Park, S., Lee, S., Lee, H., Yun, S., and Lee, B. (2021). *SPADEs: Spatial Dependency Parsing for Semi-Structured Document Information Extraction.*. doi:10.48550/arXiv.2106.14624

## 8.3 Named Entity Recognition and Span Extraction

Zaratiana, U., Tomeh, N., Holat, P., and Charnois, T. (2023). *GLiNER: Generalist Model for Named Entity Recognition using Bidirectional Transformer.doi:* https://doi.org/10.48550/arXiv.2311.08526
Model available at: huggingface.co/urchade/gliner_multi-v2.1